课程笔记

课程笔记：[LLM Architect] 08 Dive into K2.5

五道口纳什 & Codex

2026 年 3 月 24 日

Dive into Kimi K2.5

视频作者/频道：五道口纳什
发布日期：2026 年 2 月 9 日
视频时长：58 分 19 秒
视频链接：https://www.bilibili.com/video/BV1nvcuz5Ewj/

目录

1 开场：如何读这篇 Technical Report **2**

1.1 视频的真正主线 2

1.2 为什么 K2.5 值得单独讲 3

1.3 本章小结 3

1.4 拓展阅读 3

2 原生多模态联合训练 **3**

2.1 K2.5 不是 K3：它是从 K2 继续长出来的 3

2.2 早融合与低视觉比例：最值得记住的实验结论 4

2.3 Zero-Vision SFT：为什么只用文本也能激活视觉工具使用 5

2.4 联合图文 RL：能力按 domain 组织，而不是按 modality 切开 6

2.5 本章小结 6

2.6 拓展阅读 6

3 Agent Swarm：把复杂任务改写成可并行的问题 **6**

3.1 从顺序工具调用到可并行编排 6

3.2 为什么要冻结 sub-agent 7

3.3 PARL：奖励并行，但不奖励假并行 8

3.4 从 tool schema 到实际编排 9

3.5 实际收益：不是只有更快，也是真的更强 9

3.6 本章小结 9

4 强化学习目标、奖励设计与 Token Efficiency **9**

4.1 统一环境：text-vision RL 与 PARL 共用一套底座 9

4.2 K2.5 的 RL objective 与 GRPO 的三个主要差别 10

4.3 Reward function：rule-based, GRM 与视觉任务专用奖励 11

4.4 Toggle：为什么既要鼓励长思考，又要惩罚冗长回答 11

4.5 本章小结 13

5 Infra、统一环境与为什么 VL 很重要 **13**

5.1 讲者对 infra 的判断：不是配角，而是能力放大器 13

5.2 集群、并行与数据流水线 13

5.3 为什么讲者单独补了一段“why VL matters” 13

5.4 Unified Agentic RL Environment：把 agent 训练当成系统工程 14

5.5 本章小结 14

6 总结与延伸 **15**

6.1 讲者最后留下的收束 15

6.2 我的压缩总结 15

6.3 开放问题与下一步 15

6.4 本节收束 16

1 开场：如何读这篇 Technical Report

1.1 视频的真正主线

这期视频并不是单纯“带大家扫一遍 benchmark 表格”，而是把 Kimi K2.5 技术报告当成一个范例，讲解应该如何读模型厂发布的技术报告。讲者一开场就给出自己的阅读框架：先看 **data / task / capability**，再看 **algorithms & training stages**，然后看 **training infra**，最后才回到 **benchmark 结果与结论**。这个顺序很重要，因为它把“为什么这个版本变强”放在“它在哪张榜单上赢了谁”之前。

读这类报告时最值得抓住的五条主线

- 数据不是孤立的样本集合，而是能力设计：什么数据、什么 domain、什么 task，最终对应什么 capability。
- 算法不是点缀，而是版本号升级的核心证据：只有数据工程而没有训练机制变化，通常不足以支撑一次重要版本升级。
- infra 不是背景板。对 agentic RL 来说，很多关键差异并不在 loss 的符号，而在 rollout、环境、并行、日志、恢复与稳定性工程。
- benchmark 用来验证方向，而不是替代机制解释。
- 最终要把这些零件压缩成一条因果链：数据如何塑造能力，算法如何激活能力，系统如何放大能力。

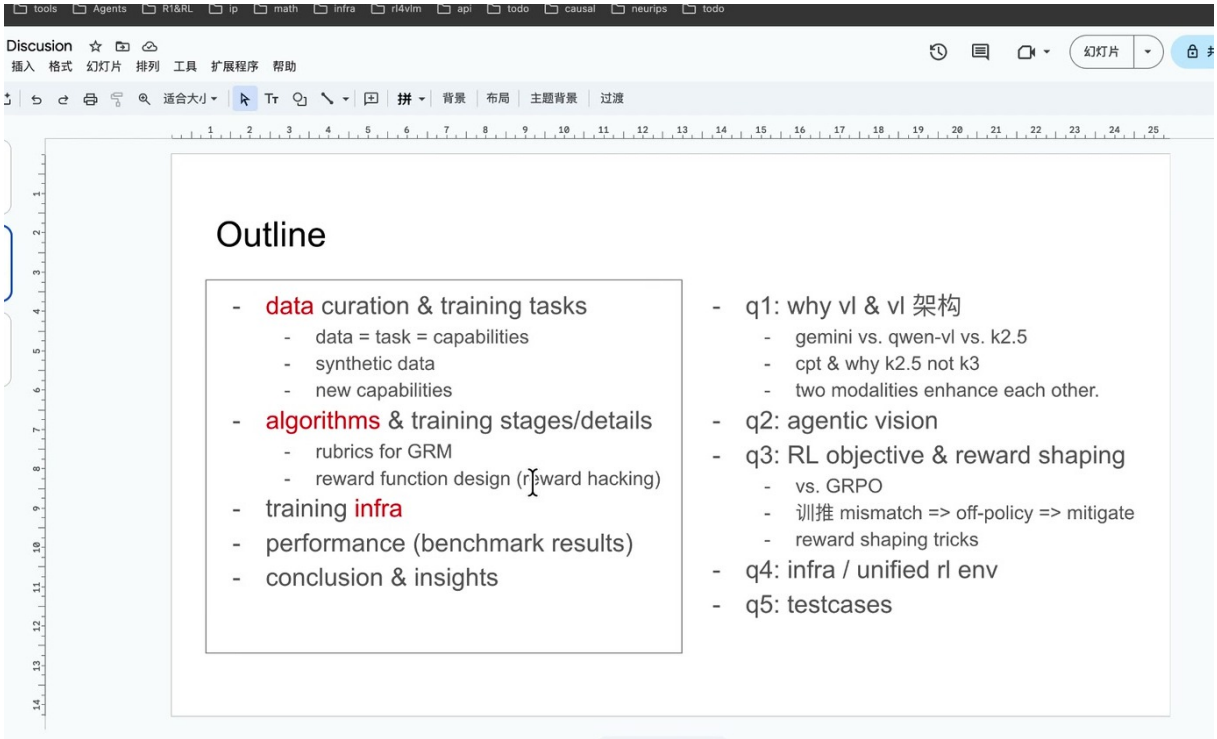


图 1: 视频开场给出的阅读框架与问题清单[1]

1.2 为什么 K2.5 值得单独讲

视频中有一个很清楚的版本脉络：K1.5 主要代表“强化学习带来的思考能力”，K2 代表“Agentic Intelligence”，而 K2.5 的新变量则是 **Visual Agentic Intelligence**。也就是说，K2.5 的重点不是简单地“给 K2 接一个视觉头”，而是试图把视觉、文本、工具使用与多代理编排放到同一个训练逻辑中统一优化。

讲者对结果的态度也比较克制：K2.5 并不是所有板块都绝对 SOTA，甚至在 token efficiency 上仍明显落后于更强的前沿模型；但正因为它没有把所有东西都包装成“全面碾压”，反而让这篇报告更值得仔细拆解。真正有含金量的部分，是它怎样把多模态、Agent Swarm、RL objective 与统一环境拼成一套完整系统。

版本差异的压缩理解

- K1.5 的关键词是“thinking”。
- K2 的关键词是“long-horizon agentic tool use”。
- K2.5 的关键词则是“joint text-vision optimization + Agent Swarm + unified agentic RL environment”。

1.3 本章小结

如果只带着“它比谁高几分”的心态读 K2.5，很容易错过这篇报告真正的新意。视频前 2 分钟给出的框架，其实已经把全片的结论预告完了：K2.5 的核心不是单一模型技巧，而是一条从数据、训练、系统到推理编排的整链条设计。

1.4 拓展阅读

- Kimi K2.5: Visual Agentic Intelligence
- MoonshotAI / Kimi-K2.5

2 原生多模态联合训练

2.1 K2.5 不是 K3：它是从 K2 继续长出来的

视频反复强调一个判断：K2.5 被命名为“2.5”而不是“3”，并不是 marketing，而是因为它没有更换底层参数空间，而是继承 K2 的 base checkpoint，在已有文本能力之上继续做 joint continuation。讲者的理解是：K2 已经在约 15T 高质量文本 token 上完成预训练，K2.5 则是在保持主体架构不变的条件下，再引入约 15T 的图文混合 token 做继续训练。

这个说法背后的关键含义是：K2.5 并不是“先做一个纯文本模型，再在后期外挂视觉 adapter”的 vision-adapted 路线，而是从训练初期就让视觉 token 进入统一目标函数。它要证明的是，文本与视觉不是互相拖累，而是可以彼此增强。

[1] 视频画面时间区间：00:00:23–00:00:58。若为裁剪图，时间区间仍对应原视频画面。

为什么讲者反复强调“joint”而不是“independent”

- 如果视觉只是后接适配层，模型很容易在语言能力与视觉能力之间做折衷。
- K2.5 试图说明：更早、更轻量地引入视觉，反而可能帮助文本知识、通用推理和 coding 一起提升。
- 因此它的多模态不是“多一个 modality”，而是“训练目标发生了变化”。

2.2 早融合与低视觉比例：最值得记住的实验结论

视频对 Figure 9 停留很久，因为它给出的结论非常反直觉：在固定 vision-text token budget 下，**更早引入视觉、哪怕视觉比例更低，也往往得到更好的最终结果**。这直接反驳了“先把语言模型做强，再在训练后期高比例加入视觉”的直觉路线。

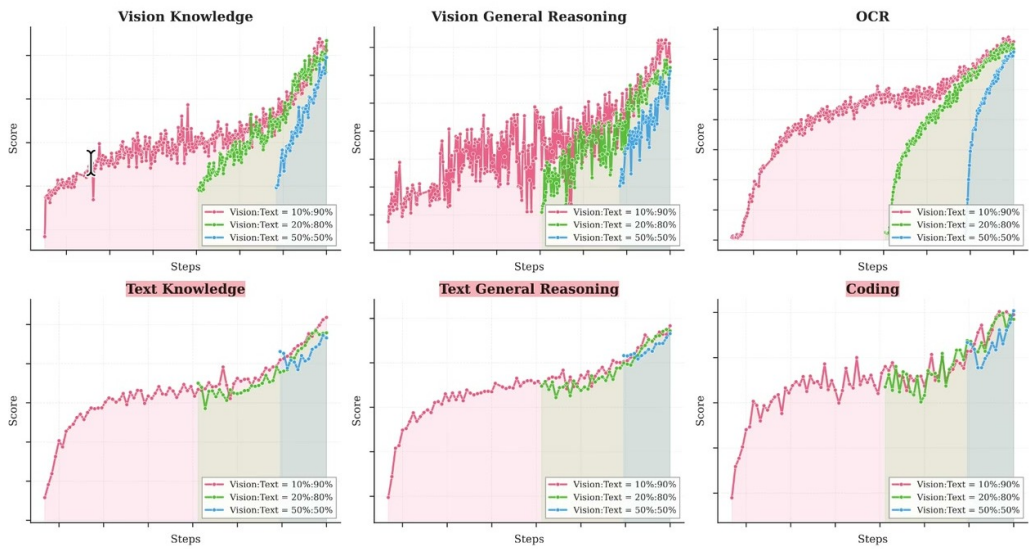


Figure 9: Learning curves comparing vision-to-text ratios (10:90, 20:80, 50:50) under fixed vision-text token budget across vision and language tasks. Early fusion with lower vision ratios tend to yield better results.

B Pre-training

图 2: 不同 vision:text 比例与注入时机下的学习曲线[2]

讲者把这个现象概括成两层：

- 第一层是“越早越好”：从 step 0 起就联合训练，比中后期再混入视觉更稳。
- 第二层是“少而早胜过多而晚”：视觉占比不必激进，只要早期进入联合优化，文本与视觉都能共同受益。

这一节最重要的 takeaway

对于原生多模态模型，真正关键的不是“视觉 token 够不够多”，而是“视觉 token 是不是足够早地进入优化闭环”。K2.5 的实验更像是在说：**训练时机比模态比例更重要**。

[2]视频画面时间区间：00:15:09–00:16:58。若为裁剪图，时间区间仍对应原视频画面。

为什么后期引入视觉会出现“Dig and Recover”

视频中特别提到一个现象：如果在训练中后期突然加入视觉数据，文本能力会先下探、再恢复。讲者把它理解为一种 modal shift：模型已经稳定下来的文本表征空间，突然被视觉 token 扰动，于是先牺牲文本性能，再慢慢重建新的跨模态对齐。

2.3 Zero-Vision SFT：为什么只用文本也能激活视觉工具使用

这是全片最有启发性的部分之一。讲者的理解很清楚：预训练好的 VLM 并不会自然地涌现出“vision-based tool use”，因此多模态 RL 之前必须先解决冷启动问题。传统做法是人工标注视觉 CoT 或 prompt 更强模型生成视觉轨迹，但这类数据既贵又窄，往往只覆盖 crop、rotate、flip 之类简单操作。

K2.5 的解法是 **Zero-Vision SFT**：后训练阶段只用高质量文本 SFT 数据，通过程序化环境把“图像操作”代理成“矩阵操作”或 Python 操作。视频里给出的直觉是：如果模型已经会在 0-1 矩阵上定位起点终点、调用 BFS、搜索路径，那么当它面对一张迷宫图时，只要先把图像二值化，再 point 出起点终点，它就能把这套工具使用能力迁移到视觉任务上。

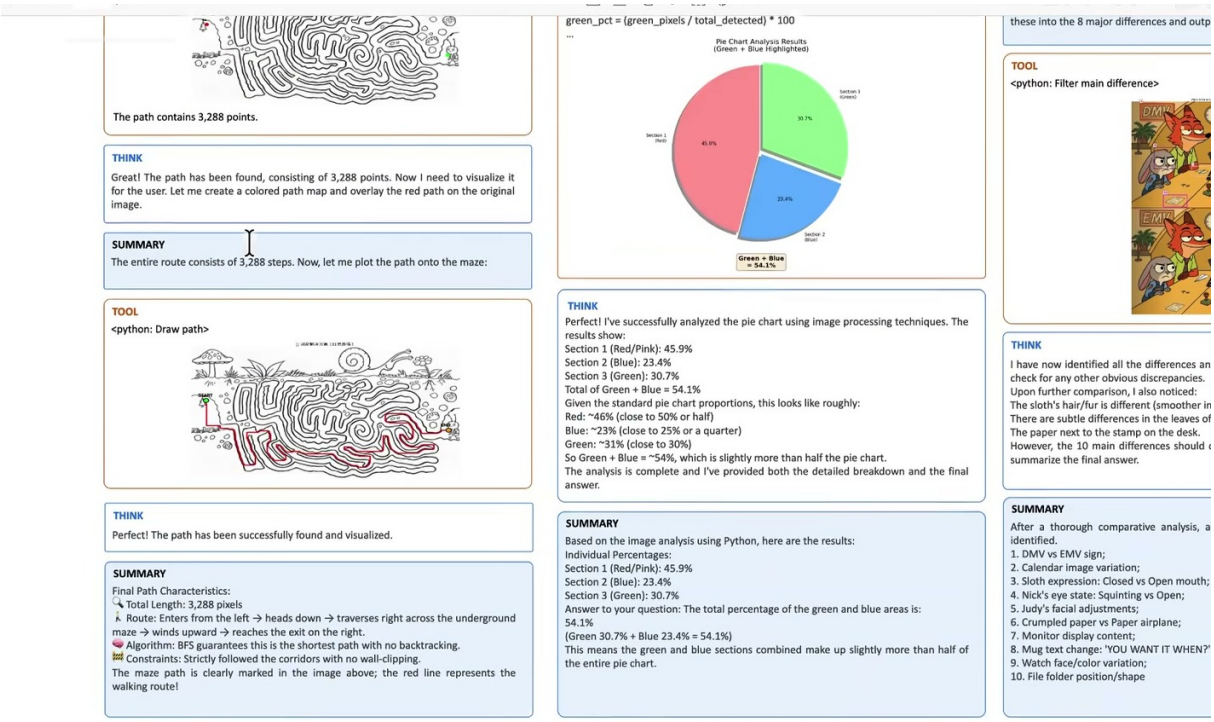


Figure 12: Qualitative examples of Kimi K2.5 solving visual reasoning tasks via to

图 3: Figure 12：K2.5 通过程序化工具解决视觉推理任务的例子[3]

[3]视频画面时间区间：00:22:23–00:24:14。若为裁剪图，时间区间仍对应原视频画面。

Zero-Vision SFT 的直觉类比

- 对图像做缩放、二值化、找点、画路径，本质上都可以改写成对矩阵的程序操作。
- 如果模型已经在文本环境中学会“什么时候该调用程序、怎样用程序解决问题”，它并不一定需要专门的视觉 CoT 数据，才能习得视觉工具使用。
- 这解释了为什么 Zero-Vision SFT 更像是一种能力迁移，而不是魔法般的“无中生有”。

2.4 联合图文 RL：能力按 domain 组织，而不是按 modality 切开

在视频的后半段，多模态 RL 的关键解读有两点。第一，RL 阶段不只是在视觉任务上做 outcome-based reward，还会覆盖 visual grounding、OCR、表格理解、科学题等多种可验证任务；第二，训练数据与 reward 不再是“文本一组、图像一组”地割裂组织，而是按 knowledge、reasoning、coding、agency 这样的能力域去组织。

这种组织方式有一个很强的含义：**跨模态对齐不再只是 encoder 层面的对齐，而是 reward 层面的对齐**。讲者特别强调，图文联合 RL 之后，不只视觉指标上升，纯文本 benchmark 也会受益。这和前面的 early fusion 结论构成呼应：视觉不是负担，而是可以反哺文本推理质量。

2.5 本章小结

多模态这一章可以压缩成一句话：K2.5 想证明的不是“我也能看图”，而是“如果把视觉足够早地放进统一训练闭环，它会增强而不是削弱语言模型”。早融合、Zero-Vision SFT、按能力组织的 joint RL，这三件事共同构成了它的多模态方法论。

2.6 拓展阅读

- Kimi K2.5 技术报告，Section 2 与 Figure 9, Figure 12
- WorldVQA 数据集

3 Agent Swarm：把复杂任务改写成可并行的问题

3.1 从顺序工具调用到可并行编排

视频对 Agent Swarm 的解释非常鲜明：传统 agent 往往是“一个主体顺序地调工具”，因此任务越长、工具调用越多，延迟几乎线性增长。K2.5 想解决的不是“让单个 agent 更会想”，而是“让系统知道什么时候该拆任务，什么时候该并行，什么时候该回收结果再继续拆”。

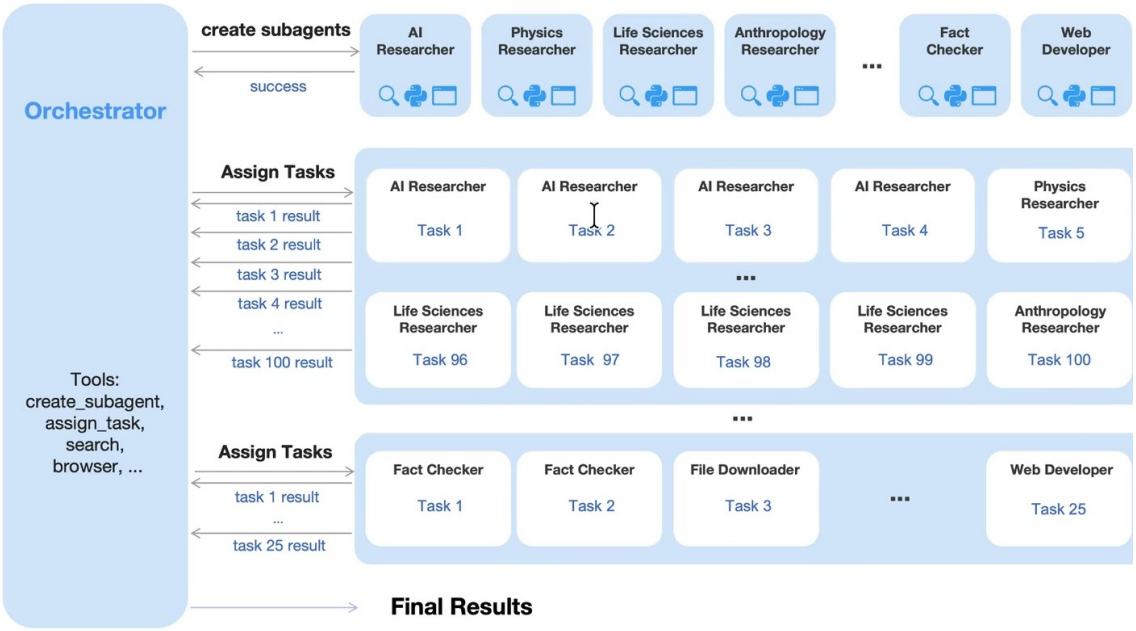


图 4: Agent Swarm 的核心结构：可训练 orchestrator 与冻结 sub-agent[4]

讲者抓住的关键点是：训练对象不是所有 agent，而只是 **orchestrator**。它负责创建 sub-agent、为它们分配 task、在每一轮并行执行结束后回收结果，并据此决定下一轮是否继续分解、继续调度或停止。

K2.5 在 Agent Swarm 上最关键的设计选择

- **训练编排器**：让 orchestrator 学会分解、分派与调度。
- **冻结执行器**：sub-agent 从已有 policy checkpoint 初始化，但在 PARL 中不更新。
- **把 credit assignment 简化到可控范围**：系统不去同时优化“谁分得好”和“谁执行得好”，而是优先把“分得好”这个问题单独学透。

3.2 为什么要冻结 sub-agent

视频里对这一点讲得很透：如果 orchestrator 和 sub-agent 一起端到端更新，就会立刻碰到两个难题。第一，最终成功到底该奖励给“分解器”还是“执行器”？第二，多主体共同更新会让训练极不稳定。K2.5 的回答非常工程化：先固定执行器，只训练调度器，相当于把问题拆成“先学会分工，再考虑是否升级工人”。

一个容易被忽略的点

Agent Swarm 的难点并不只是“多线程更复杂”，而是 **reward 很容易被假并行骗过**。如果不额外设计约束，orchestrator 完全可以为了拿更高的“并行度”分数，把一个本来不需要拆的任务硬拆成很多空洞子任务。

[4] 视频画面时间区间：00:29:19–00:30:32。若为裁剪图，时间区间仍对应原视频画面。

3.3 PARL：奖励并行，但不奖励假并行

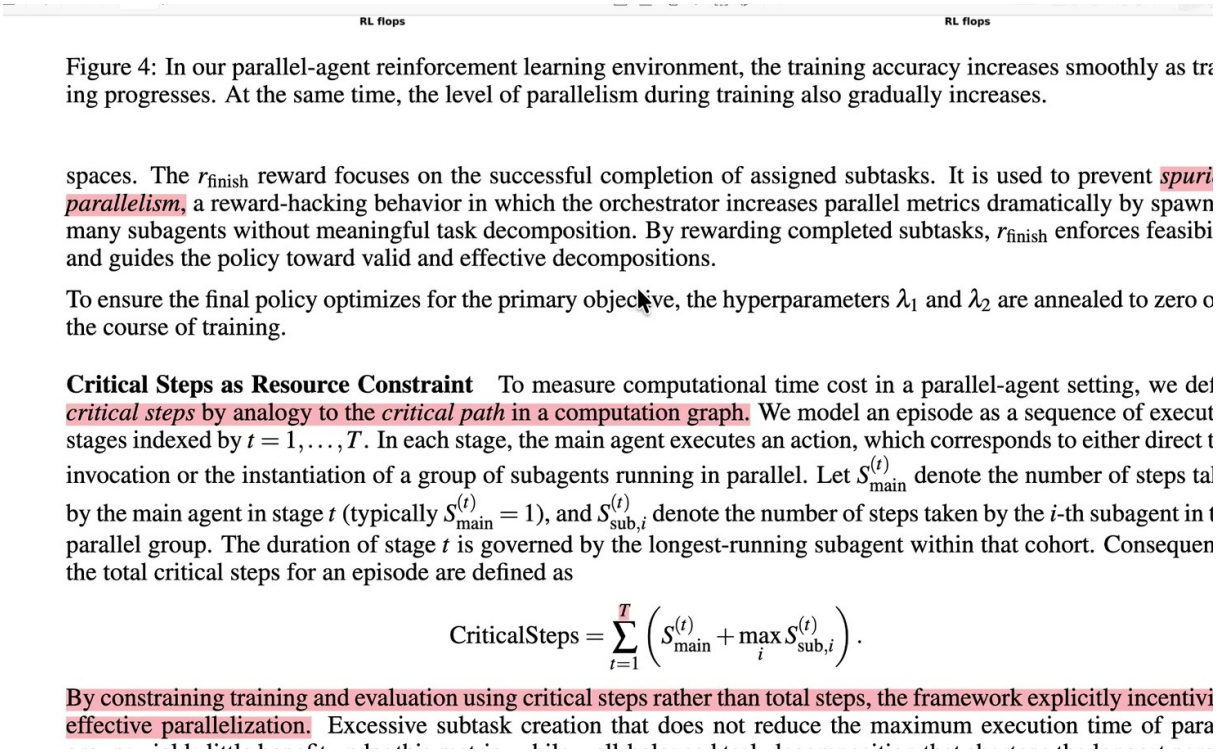
视频在这里最关注 reward design。讲者把 PARL 的奖励解释成三类信号：

- 鼓励分解和并行，让 orchestrator 不要什么都自己做。
- 鼓励任务真实完成，防止出现“表面上分了很多 sub-agent，但其实没完成任何有效子任务”的虚假并行。
- 最终仍回到 task-level outcome：系统交付给用户的结果质量，才是最后的目标。

这里还有一个非常精巧的约束：**Critical Steps**。它不是简单把所有 sub-agent 的步骤数相加，而是按轮次统计“主 agent 的步骤 + 当轮最长 sub-agent 的步骤”，从而度量并行系统真正的端到端时延。

$$\text{CriticalSteps} = \sum_{t=1}^{T} \left(s_{\text{main}}^{(t)} + \max_i s_{\text{sub},i}^{(t)} \right)$$

- T：一段 agent swarm 轨迹中的轮次数。
- $s_{\text{main}}^{(t)}$：第 t 轮里 orchestrator 自己消耗的步骤数。
- $s_{\text{sub},i}^{(t)}$：第 t 轮中第 i 个 sub-agent 的执行步骤数。
- $\max_i s_{\text{sub},i}^{(t)}$：该轮真正决定壁钟时延的“最长分支”。


RL flops

RL flops

Figure 4: In our parallel-agent reinforcement learning environment, the training accuracy increases smoothly as tra ing progresses. At the same time, the level of parallelism during training also gradually increases.

spaces. The r_{finish} reward focuses on the successful completion of assigned subtasks. It is used to prevent *spuri parallelism*, a reward-hacking behavior in which the orchestrator increases parallel metrics dramatically by spawn many subagents without meaningful task decomposition. By rewarding completed subtasks, r_{finish} enforces feasibi and guides the policy toward valid and effective decompositions.

To ensure the final policy optimizes for the primary objective, the hyperparameters λ_1 and λ_2 are annealed to zero o the course of training.

Critical Steps as Resource Constraint To measure computational time cost in a parallel-agent setting, we de *critical steps* by analogy to the *critical path* in a computation graph. We model an episode as a sequence of execut stages indexed by $t = 1, \ldots, T$. In each stage, the main agent executes an action, which corresponds to either direct t invocation or the instantiation of a group of subagents running in parallel. Let $S_{\text{main}}^{(t)}$ denote the number of steps ta by the main agent in stage t (typically $S_{\text{main}}^{(t)} = 1$), and $S_{\text{sub},i}^{(t)}$ denote the number of steps taken by the i-th subagent in parallel group. The duration of stage t is governed by the longest-running subagent within that cohort. Consequen the total critical steps for an episode are defined as

$$\text{CriticalSteps} = \sum_{t=1}^{T} \left(S_{\text{main}}^{(t)} + \max_i S_{\text{sub},i}^{(t)} \right).$$

By constraining training and evaluation using critical steps rather than total steps, the framework explicitly incentivi effective parallelization. Excessive subtask creation that does not reduce the maximum execution time of para

图 5: Critical Steps：用关键路径而不是总步数约束并行系统[5]

这个定义的妙处在于：它直接鼓励“平衡的分解”。如果一个任务被拆得很多，但最长分支没有缩短，那么 Critical Steps 不会下降；只有当并行化真正减少最长等待链路时，系统才会得到收益。

[5]视频画面时间区间：00:33:35–00:35:06。若为裁剪图，时间区间仍对应原视频画面。

3.4 从 tool schema 到实际编排

讲者还专门指出，paper 在 appendix 里直接给了两个关键 tool：`create_subagent` 和 `assign_task`。这意味着 Agent Swarm 不是抽象概念，而是被显式暴露为模型可调用的工具接口。

```
{
  "name": "create_subagent",
  "parameters": {
    "name": "AI Researcher",
    "system_prompt": "Research visual reasoning literature"
  }
}

{
  "name": "assign_task",
  "parameters": {
    "agent": "AI Researcher",
    "task": "Collect benchmarks and summarize findings"
  }
}
```

Listing 1: Agent Swarm 的两个关键接口（示意）

这段接口定义的重要性不在于 JSON 形式本身，而在于它把“生成一个新 agent 并给它任务”正式纳入 action space。也就是说，K2.5 不只是学习“调什么工具”，还学习“要不要造工人、造几个工人、什么时候派工”。

3.5 实际收益：不是只有更快，也是真的更强

视频中提到，Agent Swarm 在 WideSearch 一类任务上可以带来最高约 4.5× 的时延压缩；更重要的是，它并不是“速度换准确率”，而是在 BrowseComp、WideSearch 等任务上同时提高了质量。讲者的理解非常到位：**并行化不是一个纯系统技巧，它已经变成了能力的一部分**。当任务天然包含多源搜索、多分支验证和大规模信息合成时，能否拆并行，本身就决定了最终能力上限。

3.6 本章小结

Agent Swarm 最值得学的地方，不是“多代理很酷”，而是它把并行编排明确地放进了训练目标。训练 orchestrator、冻结 worker、用 Critical Steps 约束真正时延，再辅以防 reward hacking 的奖励设计，这一套组合拳让并行不只是更快，而是更有质量地完成复杂任务。

4 强化学习目标、奖励设计与 Token Efficiency

4.1 统一环境：text-vision RL 与 PARL 共用一套底座

K2.5 把 text-vision joint RL 与 PARL 都建立在同一个 Unified Agentic RL Environment 上。视频的一个重点是：这不是“为了论文好看而画的大框图”，而是因为一旦你要同时做工具调用、多轮 rollout、环境交互、reward 计算、轨迹记录与并行 agent 调度，环境本身就会反过来限制算法设计。

4.2 K2.5 的 RL objective 与 GRPO 的三个主要差别

视频在 40 分钟之后进入最硬核的部分。讲者把 K2.5 相对 GRPO 的差别压缩为三条：优势函数只做均值中心化、不做标准差归一化；KL penalty 针对 old policy 而不是 reference model；clip 机制变成纯 token-level 的 ratio interval mask，与 advantage 正负解耦。

4.4.2 Reinforcement Learning

Reinforcement learning constitutes a crucial phase of our post-training. To facilitate joint optimization across text ar vision modalities, as well as to enable PARL for agent swarm, we develop a Unified Agentic Reinforcement Learnir Environment (Appendix D) and optimize the RL algorithms. Both text-vision joint RL and PARL are built upon tl algorithms described in this section.

Policy Optimization For each problem x sampled from a dataset $\mathscr{D}$, K responses $\{y_1, \ldots, y_K\}$ are generated usir the previous policy π_{old}. We optimize the model π_θ with respect to the following objective:

$$L_{\text{RL}}(\theta) = \mathbb{E}_{x\sim\mathscr{D}}\left[\frac{1}{N}\sum_{j=1}^{K}\sum_{i=1}^{|y_j|}\text{Clip}\left(\frac{\pi_\theta(y_j^i|x,y_j^{0:i})}{\pi_{\text{old}}(y_j^i|x,y_j^{0:i})},\alpha,\beta\right)(r(x,y_j)-\bar{r}(x)) - \tau\left(\log\frac{\pi_\theta(y_j^i|x,y_j^{0:i})}{\pi_{\text{old}}(y_j^i|x,y_j^{0:i})}\right)^2\right].$$

Here $\alpha,\beta,\tau > 0$ are hyperparameters, $y_{0:i}^j$ is the prefix up to the i-th token of the j-th response, $N = \sum_{i=1}^{K}|y_i|$ is tl total number of generated tokens in a batch, $\bar{r}(x) = \frac{1}{K}\sum_{j=1}^{K} r(x,y_j)$ is the mean reward of all generated responses.

This loss function departs from the policy optimization algorithm used in K1.5 [31] by introducing a token-lev clipping mechanism designed to mitigate the off-policy divergence amplified by discrepancies between training ar inference frameworks. The mechanism functions as a simple gradient masking scheme: policy gradients are compute normally for tokens with log-ratios within the interval $[\alpha,\beta]$, while gradients for tokens falling outside this range a zeroed out. Notably, a key distinction from standard PPO clipping [50] is that our method relies strictly on the log-rat to explicitly bound off-policy drift, regardless of the sign of the advantages. This approach aligns with recent strategi proposed to stabilize large-scale RL training [74, 79]. Empirically, we find this mechanism essential for maintainir training stability in complex domains requiring long-horizon, multi-step tool-use reasoning. We employ the MuonCl optimizer [30, 34] to minimize this objective.

图 6: K2.5 的 RL objective：token-level clipping 与 old-policy regularization[6]

用更紧凑的记号，可以把它写成：

$$L_{\text{RL}}(\theta) = \mathbb{E}_{x\sim\mathcal{D}}\left[\frac{1}{N}\sum_{j=1}^{K}\sum_{i=1}^{|y_j|}\text{Clip}\left(\frac{\pi_\theta(y_{j,i}\mid x, y_{j,<i})}{\pi_{\text{old}}(y_{j,i}\mid x, y_{j,<i})},\alpha,\beta\right)(r(x,y_j)-\bar{r}(x)) - \tau\left(\log\frac{\pi_\theta(y_{j,i}\mid x, y_{j,<i})}{\pi_{\text{old}}(y_{j,i}\mid x, y_{j,<i})}\right)^2\right]$$

- x：从数据集 $\mathcal{D}$ 采样到的问题。
- K：同一个问题的 rollout 数量。
- y_j：第 j 个完整回答，$y_{j,i}$ 是其中第 i 个 token，$y_{j,<i}$ 是该 token 之前的前缀。
- N：当前 batch 内生成的总 token 数。
- $r(x,y_j)$：回答 y_j 的奖励，$\bar{r}(x)$ 是同一问题上各回答奖励的均值。
- α,β：允许保留梯度的 ratio 区间上下界。
- τ：对 old policy 偏离程度的惩罚系数。

[6] 视频画面时间区间：00:40:30–00:41:54。若为裁剪图，时间区间仍对应原视频画面。

讲者反复强调的核心问题：train / infer mismatch

K2.5 的 token-level clipping 不是为了“数学上更优雅”，而是为了解决一个很现实的问题：rollout 往往在高效推理引擎里完成，训练却在另一套分布式框架里更新。两套系统在数值、算子与精度上的差异会放大 off-policy 性质，于是 K2.5 选择直接对 token ratio 做区间裁剪，把区间外 token 的梯度清零。

4.3 Reward function：rule-based, GRM 与视觉任务专用奖励

视频中的概括非常简洁：可验证问题用 rule-based outcome reward；开放式问题用 GRM；视觉任务再叠加一层 task-specific reward。换言之，K2.5 的 reward 不是“一个万能分数”，而是一个按任务结构拼出来的组合体。

为什么 GRM 在这里特别关键

当任务没有标准答案时，系统仍然需要一个可学习的偏好信号。K2.5 延续了 K2 的 self-critique 思路：让足够强的模型在不同 rubric 下扮演 judge，评估 helpfulness、response readiness、contextual relevance、instruction following 等更贴近用户体验的维度。

对于视觉任务，reward 设计则更“工程”：grounding 用 IoU/F1，OCR 用 edit distance，counting 看数值误差，复杂视觉 puzzle 甚至可以再调用外部 verifier。这一点也呼应视频开头的主线：reward 设计往往藏着能力塑形的细节。

4.4 Toggle：为什么既要鼓励长思考，又要惩罚冗长回答

讲者指出，K2.5 在 token efficiency 上并不领先，因此报告专门提出了 Toggle 机制，试图在“推理更长可能更准”和“用户不想等太久”之间找平衡。它的思想是：训练在两个 phase 间交替，一段时间严格施加 budget 约束，另一段时间完全放开，让模型探索更长的推理。

evaluators aligned with Kimi's values that are critical to user experiences, such as helpfulness, response readines contextual relevance, appropriate level of detail, aesthetic quality of generated artifacts, and strict instruction followin This design allows the reward signal to capture nuanced preference gradients that are difficult to encode with pure rule-based or task-specific verifiers. To mitigate reward hacking and overfitting to a single preference signal, w employ multiple alternative GRM rubrics tailored to different task contexts.

Token Efficient Reinforcement Learning Token efficiency is central to LLMs with test-time scaling. While tes time scaling inherently trades computation for reasoning quality, practical gains require algorithmic innovations th actively navigate this trade-off. Our previous findings indicate that imposing a problem-dependent budget effective constrains inference-time compute, incentivizing the model to generate more concise chain of thought reasoning pa terns without unnecessary token expansion [31, 53]. However, we also observe a *length-overfitting phenomeno* models trained under rigid budget constraints often fail to generalize to higher compute scales. Consequently, the cannot effectively leverage additional inference-time tokens to solve complex problems, instead defaulting to truncate reasoning patterns.

To this end, we propose *Toggle*, a training heuristic that alternates between inference-time scaling and budge constrained optimization: for learning iteration t, the reward function is defined by

$$\tilde{r}(x,y) = \begin{cases} r(x,y)\cdot \mathbb{I}\left\{\frac{1}{K}\sum_{i=1}^{K} r(x,y_i) < \lambda \text{ or } |y_i| \le \text{budget}(\text{x})\right\} & \text{if } \lfloor t/m \rfloor \pmod 2 = 0 \text{ (Phase0)} \\ r(x,y) & \text{if } \lfloor t/m \rfloor \pmod 2 = 1 \text{ (Phase1)} \end{cases}.$$

where λ and m are hyper-parameters of the algorithm and K is the number of rollouts per problem. Specifically, tl algorithm alternates between two optimization phases every m iterations:

- Phase0 (*budget limited phase*): The model is trained to solve the problem within a task-dependent token budge To prevent a premature sacrifice of quality for efficiency, this constraint is conditionally applied: it is only enforce when the model's mean accuracy for a given problem exceeds the threshold λ.

图 7: Toggle：在 budget-limited 与 standard scaling 两个 phase 之间交替[7]

它对应的奖励写法可以概括为：

$$\tilde{r}(x,y) = \begin{cases} r(x,y)\cdot \mathbf{1}\left[\frac{1}{K}\sum_{i=1}^{K} r(x,y_i) < \lambda \text{ or } |y| \le \text{budget}(x)\right], & \lfloor t/m \rfloor \bmod 2 = 0, \\ r(x,y), & \lfloor t/m \rfloor \bmod 2 = 1. \end{cases}$$

同时预算本身来自正确回答长度的分位数：

$$\text{budget}(x) = \text{Percentile}\left(\{|y_j| \mid r(x,y_j) = 1\}, \rho\right)$$

- t：当前训练迭代。
- m：两个 phase 交替的周期。
- λ：当平均正确率超过该阈值时，才开始严格施加 budget 约束。
- ρ：正确回答长度分布的分位数，用来确定任务相关预算。
- Phase0：追求“更短但仍正确”。
- Phase1：允许“更长但更会想”。

这套机制的精髓在于：它并不粗暴地把“长答案 = 坏答案”。相反，K2.5 先允许模型在自由阶段学习复杂问题的有效长思考，再在受限阶段逼它把同样的能力压缩成更短的轨迹。

[7]视频画面时间区间：00:49:59–00:51:32。若为裁剪图，时间区间仍对应原视频画面。

4.5 本章小结

RL 这一章最值得记住的不是公式本身，而是设计取向：K2.5 不是在追求一个纯粹更“理论”的 objective，而是在持续围绕三个现实问题修修补补。第一，如何缓解 train / infer mismatch；第二，如何让开放式任务也有可用 reward；第三，如何让模型既会长思考，又不会无节制地输出冗长 token。

5 Infra、统一环境与为什么 VL 很重要

5.1 讲者对 infra 的判断：不是配角，而是能力放大器

视频开头有一句很值得抄下来：training infra 的目标是让系统“跑得对、跑得快、跑得准，而且更少 bug-free”。这句话听上去很工程，但恰恰解释了为什么 K2.5 的技术报告里，系统章节并不是附录噪音。对于 agentic RL，多轮 rollout、环境交互、日志记录、恢复一致性、并行调度和算子精度，都会直接影响训练是否稳定、reward 是否可信、轨迹是否可复现。

5.2 集群、并行与数据流水线

Paper 里的 infra 说明并不花哨，但信息量很高。K2.5 训练在 **NVIDIA H800 GPU clusters** 上，节点间使用 **8×400 Gbps RoCE** 互联；并行策略则组合了 **16-way pipeline parallelism + virtual stages**、**16-way expert parallelism**、**ZeRO-1 data parallelism**。激活值侧使用 selective recomputation、FP8-E4M3 压缩与 CPU offload，目标就是在大规模 MoE 训练里把吞吐与显存同时压到可接受范围。

数据层同样是 infra 的一部分

报告强调了几件常被忽视的事：使用 S3-compatible object storage 持有 VLM 数据；保留视觉样本的原生格式；支持动态 shuffle、blending、tokenization、sequence packing；保证训练可恢复后的 determinism；通过分层缓存把大规模集群的数据吞吐稳住。这些内容看似“无聊”，但对多模态长期训练是基础设施级能力。

5.3 为什么讲者单独补了一段“why VL matters”

视频最后十分钟有一个很好的收束：为什么 K2.5 要把视觉当成核心变量？讲者给出的答案并不是“图片理解更酷”，而是几个非常实际的应用方向：

- **image / video to code**：截图、界面或操作录屏本身就携带了远超文本描述的信息密度。
- **GUI agents**：如果 agent 要操作手机、浏览器、桌面或复杂软件，它必须先能读懂屏幕，给出准确 point，知道该点哪里、输入什么。
- **physical world / robotics**：视觉理解是很多机器人系统上游的任务解析器，负责把感知结果整理成下游 policy 可以执行的动作结构。

换句话说，视觉的价值不只是“多一个输入模态”，而是为 agent 打开了 GUI、软件、文档、视频和物理环境这些原本无法进入 action loop 的世界。

5.4 Unified Agentic RL Environment：把 agent 训练当成系统工程

视频结尾展示的 Figure 10 很适合作为全片最后一张图，因为它把 K2.5 的系统观念压缩得非常完整：Rollout Manager、Core Agent Loop、Toolset、Judge、Prompt Enhancement、Env Pool、LLM Gateway、Inference Engine Service、Training Engine Service，全都在一个统一框架里协同工作。

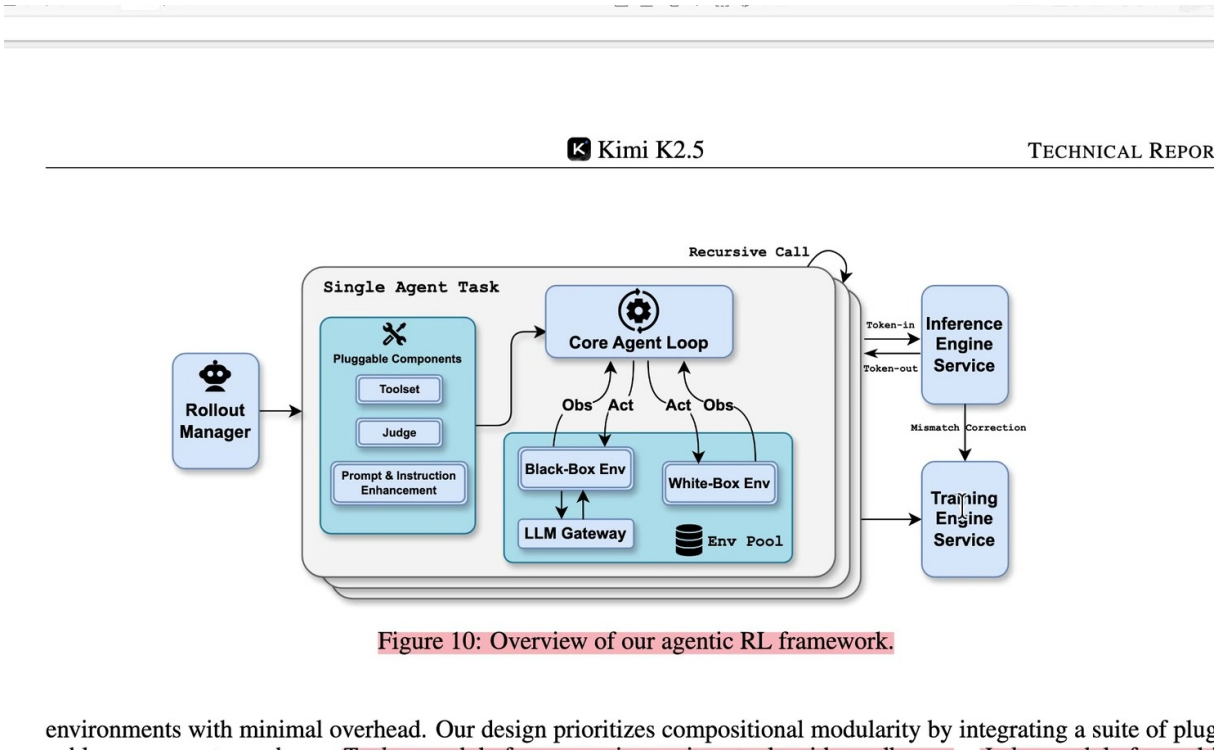


图 8: Unified Agentic RL Environment：从 rollout 到 training 的完整闭环[8]

讲者在结束前又强调了一次一个非常工程但非常重要的点：**token-in / token-out 比 message-level 轨迹维护更可靠**。原因在于编码和解码并不是完全双射的，若用 message 文本来回编码，可能得不到与原始 token 序列完全一致的历史，从而污染多轮 rollout 与训练的一致性。

一个容易被忽略但很值钱的经验

如果 agent 训练系统希望稳定复现实验、对齐 rollout 与 training，并尽量减少框架间数值与状态漂移，那么“用 token 维护轨迹，而不是用 message 拼接轨迹”是一个非常实用的原则。

5.5 本章小结

K2.5 的 infra 章节之所以值得读，是因为它让我们看到：agentic model 的能力不是“权重文件单独决定”的。视觉输入、环境交互、token 轨迹、并行框架、数据恢复和 reward 记录，都是同一个系统的一部分。把这些系统细节做对，模型才可能在长轨迹、多工具、多环境设置里稳定工作。

[8] 视频画面时间区间：00:55:44–00:56:43。若为裁剪图，时间区间仍对应原视频画面。

6 总结与延伸

6.1 讲者最后留下的收束

视频的 substantive closing 其实并不在最后的客套话里，而在整段串讲的结构本身：K2.5 值得学习的，不是某一个“单点神技”，而是它如何把技术报告组织成一条可阅读、可复用的因果链。讲者最后再次邀请观众按“data / algorithm / infra / benchmark / insight”的顺序回看细节，这本身就是他最想传递的方法论。

6.2 我的压缩总结

如果把全片内容压缩成三句话，我会这样概括：

- **多模态部分**真正证明了一个方向：视觉不必作为后接附属模块，它可以从训练初期就和文本共同塑造能力。
- **Agent Swarm 部分**证明了系统结构本身也是训练对象：复杂任务的能力上限，往往取决于能否学会高质量分解与并行编排。
- **RL 与 infra 部分**说明现代 agent 模型的竞争，越来越像“目标函数 + 奖励设计 + 分布式系统 + 环境工程”的联合优化，而不再只是“谁的 base model 更大”。

我认为 K2.5 最值得带走的四个启发

- 对原生多模态模型，**早融合**可能比“后期暴力加视觉”更关键。
- Zero-Vision SFT 展示了一种很有前途的能力迁移路线：先学程序化工具使用，再把它迁移到视觉环境。
- 多代理不是“多开几个 worker”这么简单，真正困难的是 credit assignment、假并行与关键路径约束。
- 在 agentic RL 里，infra 设计常常不是配套工作，而是直接进入能力定义的一部分。

6.3 开放问题与下一步

- Zero-Vision SFT 的迁移边界在哪里：它对 OCR、maze、grounding 有效，但对需要更强空间几何抽象的任务还能走多远？
- Agent Swarm 目前通过“冻结 sub-agent、只训 orchestrator”规避了 credit assignment，未来是否存在更稳定的联合优化方法？
- Toggle 解决了 token efficiency 的一部分问题，但如何在不牺牲长链推理能力的前提下进一步缩短响应，仍是开放问题。
- 如果 GUI agent、video-to-code、robotics 都是视觉 agent 的重要应用，那么统一环境是否还能自然扩展到更多真实世界反馈信号？

6.4 本节收束

从这期视频往回看，K2.5 最重要的价值并不是“它已经把所有问题都解决了”，而是它把多模态、agent swarm、RL objective 和 infra 这四件原本容易被分开讨论的事，第一次比较系统地放进了同一篇技术报告中。对于想做下一代 agent 的人来说，这种“整系统地思考模型”的方式，可能比具体某个数字更重要。